

Swiss Joint Master in Computer Science

13081/43081 Systems Verification

Linear-time Temporal Logic

Prof. Ulrich Ultes-Nitsche

Department of Informatics, University of Fribourg

Disadvantage of never claims

- ▶ Never claims always formulate the opposite of what we want the system to achieve.
- ▶ That can be quite counter-intuitive:
 - ▶ we want: “the systems attempts to establish a connection until it eventually establishes one”.
 - ▶ in a never claim, we express: “it is not the case that the systems always attempts to establish a connection without eventually establishing any or that the system stops attempting to establish a connection before a connection is established”
- ▶ The example above shows that never claims can be a clumsy way to express properties.

Formulating Properties Positively: *Temporal Logic*

Temporal logic can be used to describe behavioural properties over time.

- ▶ here “time” means “chronological order”
- ▶ temporal logic is an excellent formalism to describe properties
- ▶ verification is the main application area of temporal logic in computer science

We are going to use linear-time temporal logic.

Syntax of Linear-time Temporal Logic (LTL)

LTL-formulas are built from atomic propositions such as

- ▶ $x \geq 42$
- ▶ `process[p1]@critical`
- ▶ p , for some variable p

Each atomic proposition is therefore an LTL-formula.

Syntax of Linear-time Temporal Logic (LTL)

If f_1 and f_2 are LTL-formulas, then so are: (note that this is an inductive definition)

- ▶ $\neg f_1$
- ▶ $f_1 \wedge f_2$
- ▶ $f_1 \vee f_2$
- ▶ $f_1 \rightarrow f_2$

- ▶ Xf_1 "next time"
- ▶ $\Box f_1$ "always"
- ▶ $\Diamond f_1$ "eventually"
- ▶ $f_1 U f_2$ "until"

▶ (f_1)

Linear-time Temporal Structures

- ▶ LTL-formulas are interpreted over infinite sequences (of states) $x = s_0s_1s_2 \dots$
 - ▶ $x_{(i\dots)}$ represents $s_is_{i+1}s_{i+2} \dots$
 - ▶ i.e. $x_{(0\dots)} = x$, $x_{(1\dots)} = s_1s_2s_3 \dots$, etc.
- ▶ in addition, there is a labelling $\lambda(s_i)$ of state s_i with the set of those atomic propositions that are true in s_i
- ▶ the truth of LTL-formula f is then defined with respect to such a so-called *linear-time temporal structure*
 - ▶ we write " $x \models f$ " to say that " x satisfies f "

Semantics of LTL

$$x = s_0 s_1 s_2 s_3 \dots$$

For LTL-formulas f , f_1 , f_2 and atomic propositions p :

$$x \models p \quad \text{iff } p \in \lambda(s_0)$$

$$x \models \neg f \quad \text{iff it is not the case that } x \models f$$

$$x \models f_1 \wedge f_2 \quad \text{iff } x \models f_1 \text{ and } x \models f_2$$

$$x \models f_1 \vee f_2 \quad \text{iff } x \models f_1 \text{ or } x \models f_2$$

$$x \models f_1 \rightarrow f_2 \quad \text{iff } x \models f_1 \text{ implies } x \models f_2$$

$$x \models Xf \quad \text{iff } x_{(1\dots)} \models f$$

$$x \models \Box f \quad \text{iff } \forall i \geq 0 : x_{(i\dots)} \models f$$

$$x \models \Diamond f \quad \text{iff } \exists i \geq 0 : x_{(i\dots)} \models f$$

$$x \models f_1 U f_2 \quad \text{iff } \exists i \geq 0 : x_{(i\dots)} \models f_2 \wedge \forall j, i > j \geq 0 : x_{(j\dots)} \models \underline{f_1}$$

("iff" abbreviates "if and only if")

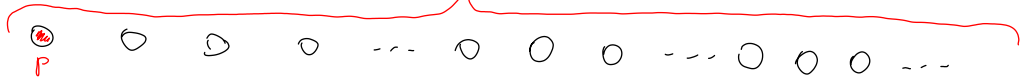
$$(p \vee \neg p) = \text{true}$$

true

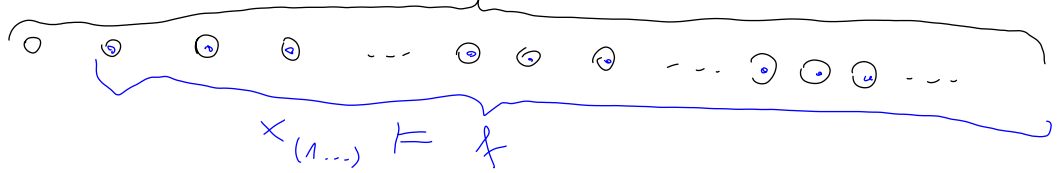
true

$$x = s_0 \quad s_1 \quad s_2 \quad s_3 \quad \dots \quad s_i \quad s_{i+1} \quad s_{i+2} \quad \dots \quad s_j \quad s_{j+1} \quad s_{j+2} \quad \dots$$

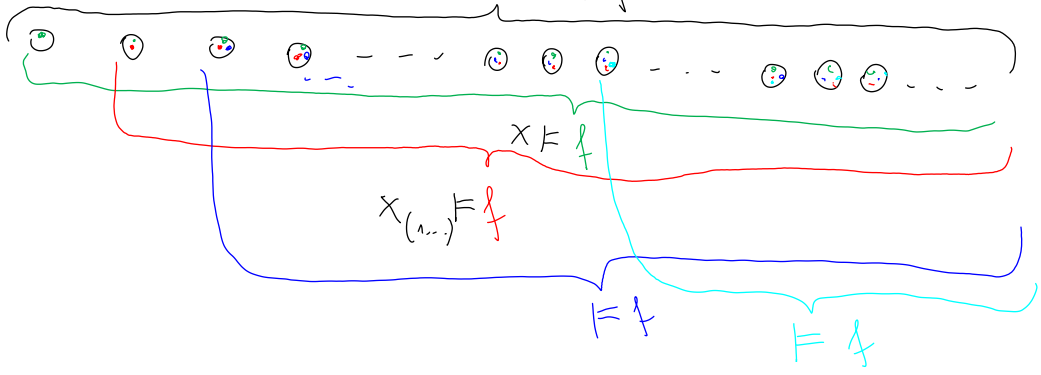
P : atomic prop. $x \models P$



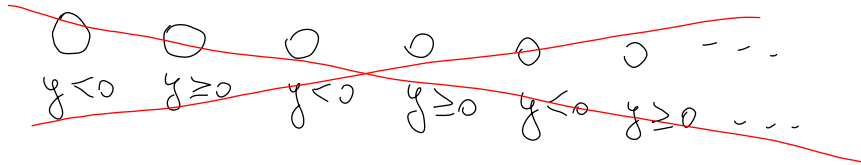
$$x \models X \neq$$



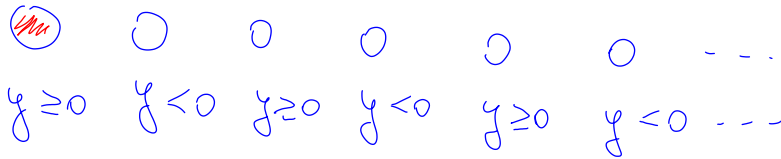
$$x \models \Box \neq$$

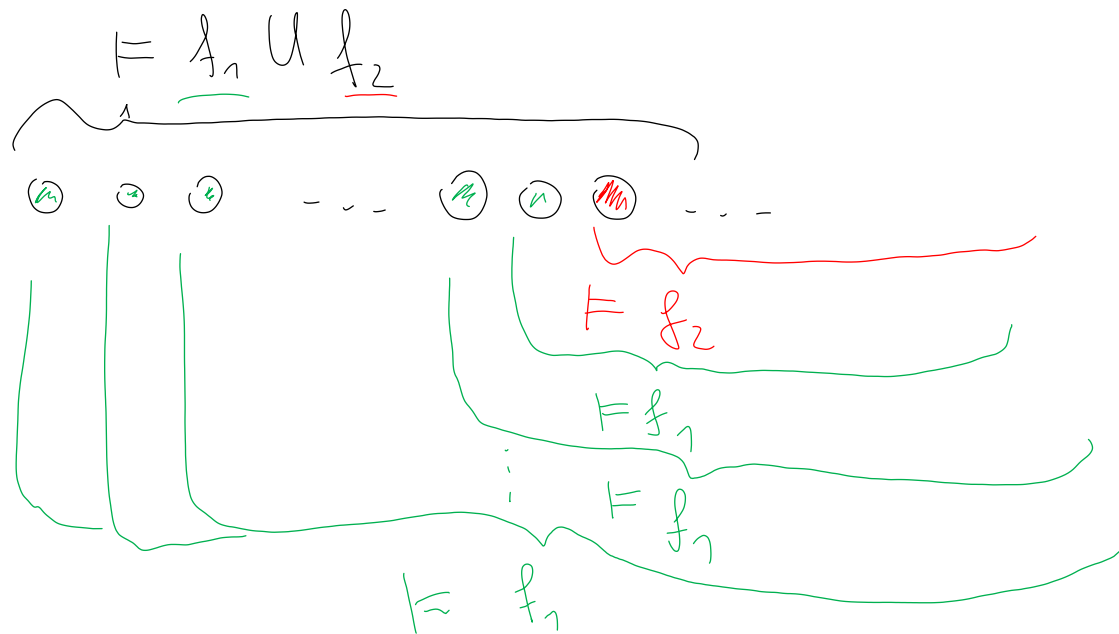
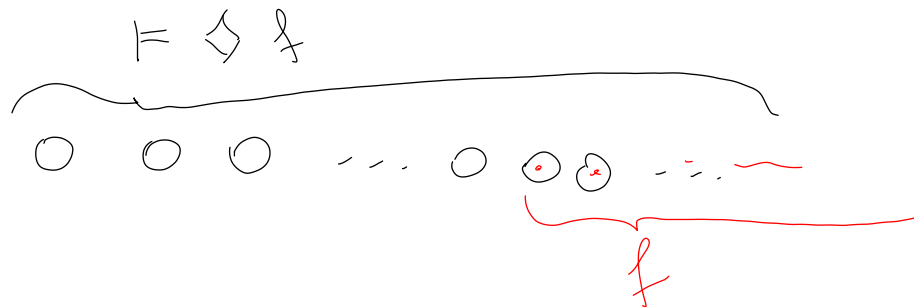


$$f = \left(\Box \left((y < 0 \rightarrow X(y \geq 0)) \wedge (y \geq 0 \rightarrow X(y < 0)) \right) \right) \wedge y = 0$$



$y = 0$





Particular Operators

The following operators are just abbreviations for particular operator combinations. Can you guess what these are?

- ▶ $f_1 \vee f_2 \equiv \neg(\neg f_1 \wedge \neg f_2)$ *de Morgan's Law*
- ▶ $f_1 \rightarrow f_2 \equiv \neg f_1 \vee f_2 \equiv \neg(\neg f_1 \wedge \neg f_2)$
- ▶ $\Diamond f \equiv \text{true} \cup f$
- ▶ $\Box f \equiv \neg \Diamond \neg f \equiv \neg(\text{true} \cup (\neg f))$

$\neg, \wedge, \vee, \cup$ "bare minimum"

Two more Operators (for completeness reasons, we won't use them)

Weak Until (W):

$$\blacktriangleright f_1 W f_2 \equiv (f_1 U f_2) \vee (\Box f_1)$$

Release (V):

$$\blacktriangleright f_1 V f_2 \equiv \neg((\neg f_1) U (\neg f_2))$$

Satisfiability and Validity

- ▶ an LTL-formula f is *satisfiable* if and only if there exists a linear-time temporal structure (x, λ) such that $x \models f$
- ▶ an LTL-formula f is *valid* if and only if for all linear-time temporal structures (x, λ) it holds $x \models f$

How can we express validity using satisfiability?

f is satisfiable $\Leftrightarrow \neg f$ is not valid

f is valid $\Leftrightarrow \neg f$ is not satisfiable

A Question for You

$\forall x : x \models \Box f \rightarrow \Diamond f \quad (\Rightarrow)$
 $| x \models \Box f \text{ implies } x \models \Diamond f$
 $(\Rightarrow)$
 $\forall i \geq 0 : x_{(i...)} \models f \Rightarrow$
 $\Box f \rightarrow \Diamond f \quad \exists i \geq 0 : x_{(i...)} \models f$
 formulas f ?
 $(\Rightarrow)$
 $x \models \Diamond f$

How could one show that

is a valid LTL-formula, for all LTL-formulas f ?

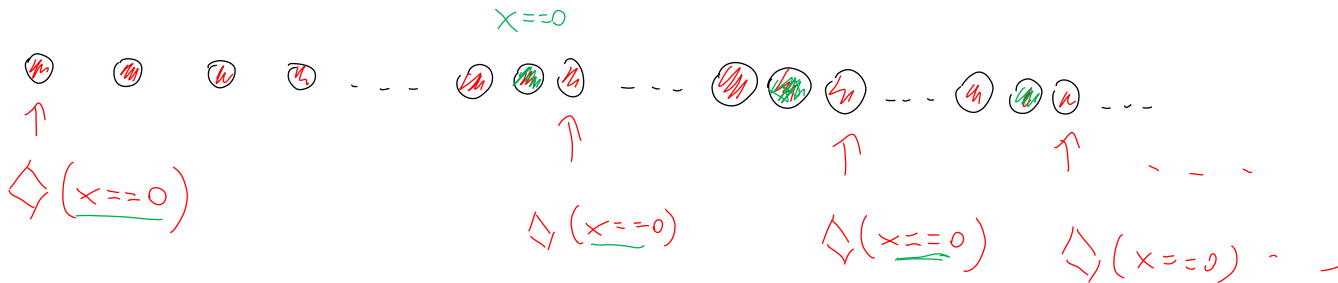
Another Question for You

$\Box \Diamond \equiv \text{infinitely often}$

What could be the meaning of LTL-formula

$$\Box \Diamond (x == 0)$$

?



Spin can check LTL-formulas against Promela models

Instead of a never claim, we may write:

```
ltl [name] { some formula }
```

For instance:

```
1      /* erroneous solution of mutex */
2
3      byte my_turn;
4
5      active proctype process1()
6      {
7          my_turn==0;
8          my_turn=1;
9          critical: my_turn = 0;
10     }
11
12     active proctype process2()
13     {
14         my_turn==0;
15         my_turn=1;
16         critical: my_turn = 0;
17     }
18
19     ltl mutex {[] (!(process1@critical && process2@critical))}
```

MutExErr6.pml



Another Inlined LTL Example

```
1  #define Aturn false
2  #define Bturn true
3
4  bool x, y, t;
5
6  active proctype A() {
7      do
8          :: x = true;
9          t = Bturn;
10         (y == false || t == Aturn)
11         criticalA: x = false;
12     od
13 }
14
15 active proctype B() {
16     do
17         :: y = true;
18         t = Aturn;
19         (x == false || t == Bturn)
20         criticalB: y = false;
21     od
22 }
23
24 ltl mutex {[] (!(A@criticalA && B@criticalB))}
```

MutExCorr9.pml (see also: **MutExCorr10.pml**)

What does Spin do with an LTL-formula?

```
#define p      x==0
#define q      y==0
ltl {[] ((p -> ! q) && (q -> ! p))}
```

What does Spin do with an LTL-formula?

```
#define p      x==0
#define q      y==0
ltl {[] ((p -> ! q) && (q -> ! p))}
```

it (basically) constructs a never claim:

```
never { /* !([] ((p -> ! q) && (q -> ! p))) */
T0_init:
    do
        :: atomic { ((p) && (q)) -> assert (!((p) && (q))) }
        :: (1) -> goto T0_init
    od;
accept_all:
    skip
}
```

What does Spin do with an LTL-formula?

```
#define p      x==0
#define q      y==0
ltl {[] ((p -> ! q) && (q -> ! p))}
```

it (basically) constructs a never claim:

```
never { /* !([] ((p -> ! q) && (q -> ! p))) */
T0_init:
    do
        :: atomic { ((p) && (q)) -> assert (!((p) && (q))) }
        :: (1) -> goto T0_init
    od;
accept_all:
    skip
}
```

We can use the “-f” option of the command-line interface.

What does Spin do with an LTL-formula? (a less recent attempt)

```
#define p      x==0
#define q      y==0
ltl {[] ((p -> ! q) && (q -> ! p))}
```

it (basically) constructs a never claim:

```
never { /* !([] ((p -> ! q) && (q -> ! p))) */
T0_init:
    if
        :: ((p) && (q)) -> goto accept_all
        :: (1) -> goto T0_init
    fi;
accept_all:
    skip
}
```

We can use the “-f” option of the command-line interface.

How does Spin do that?

- ▶ If LTL-formula f describes a property that a Promela model should satisfy, then all of its behaviours must satisfy f .
- ▶ Consequently, none of its behaviours should satisfy $\neg f$ (as with never claims).
- ▶ To construct a never claim to f , Spin uses $\neg f$, possibly in a simplified version, and translates it into a process description that becomes the never claim.

That's all

Do you have questions?